TRƯỜNG ĐẠI HỌC SÀI GÒN

KHOA CÔNG NGHỆ THÔNG TIN

BÁO CÁO BÀI TẬP LỚN

MÔN CÔNG NGHỆ PHẦN MỀM

BrewLite

Ứng dụng đặt đồ uống và thanh toán không tiền mặt

Giảng viên hướng dẫn: Từ Lãng Phiêu

NHÓM 7

STT	Họ và tên	MSSV
1	Phan Anh Khang	3124410145
2	Đoàn Quang Khải	3124410138
3	Nguyễn Huy Hoàng	3124410105
4	Huỳnh Thái Kiệt	3124410172
5	Đỗ Mạnh Huy	3124410113

TP. HỒ CHÍ MINH, THÁNG 11 NĂM 2026

Lời cảm ơn

Nhóm 7 xin cảm ơn thầy Từ Lãng Phiêu đã giảng dạy và hướng dẫn nhóm trong môn Công nghệ phần mềm. Quá trình thực hiện BrewLite giúp nhóm vận dụng phân tích yêu cầu, thiết kế kiến trúc, lập trình, kiểm thử và phối hợp công việc theo Scrum vào một sản phẩm có luồng nghiệp vụ cụ thể.

Nhóm xin cảm ơn các thành viên đã cùng thảo luận yêu cầu, xây dựng chức năng và kiểm tra các phần tích hợp. Nhóm mong nhận được góp ý để hoàn thiện hệ thống và chất lượng báo cáo.

Ghi chú về phạm vi báo cáo. Nội dung được đối chiếu với mã nguồn và tài liệu trong kho BrewLite tại ngày 03/11/2026. Danh mục yêu cầu mô tả hành vi cần đạt; hiện trạng triển khai được nêu riêng ở Chương 4 và Chương 5. Các kết quả kiểm thử có số liệu được trích từ hồ sơ kiểm chứng đã lưu trong dự án, không phải kết quả chạy lại trong lúc soạn báo cáo.

Mục lục

Lời cảm ơn **i**

1 Giới thiệu đề tài **1**

1.1 Bối cảnh và lý do chọn đề tài 1

1.2 Mục tiêu 1

1.3 Phạm vi và đối tượng sử dụng 2

1.4 Công nghệ sử dụng 2

1.5 Cơ sở xây dựng báo cáo 3

2 Phân tích yêu cầu **4**

2.1 Quy ước và mức ưu tiên 4

2.2 Danh sách User Story và tiêu chí chấp nhận 4

2.3 Yêu cầu chức năng (Functional Requirements) 9

2.4 Yêu cầu phi chức năng (Non-functional Requirements) 10

2.5 Quy tắc nghiệp vụ 10

2.6 Đặc tả luồng tạo đơn và thanh toán 11

3 Thiết kế hệ thống **12**

3.1 Kiến trúc tổng thể 12

3.2 Phân chia trách nhiệm backend 12

3.3 Thiết kế cơ sở dữ liệu 13

3.4 Máy trạng thái đơn hàng 15

3.5 Thiết kế API hiện có 15

3.6 Thiết kế các màn hình chính 16

4 Hiện thực và đối chiếu mã nguồn **17**

4.1 Tổ chức mã nguồn 17

4.2 Các điểm hiện thực chính 17

4.3 Ma trận yêu cầu và hiện trạng 18

4.4 Cấu hình và triển khai 19

5 Kiểm thử và đánh giá **20**

5.1 Chiến lược kiểm thử 20
5.2 Các ca kiểm thử tiêu biểu 20
5.3 Kết quả từ hồ sơ kiểm chứng 21
5.4 Vấn đề và giới hạn cần xử lý 22
5.5 Điều kiện nghiệm thu 22

6 Tổ chức thực hiện và kết luận **23**

6.1 Thành viên và vai trò Scrum 23
6.2 Bảng phân công công việc 23
6.3 Product Backlog và kế hoạch Sprint 25
6.4 Kết quả đạt được 26
6.5 Hạn chế và hướng hoàn thiện 26

Tài liệu tham khảo **27**

A Hướng dẫn chạy và phân công chi tiết **28**

A.1 Chạy ứng dụng trên môi trường phát triển 28
A.2 Lệnh kiểm tra ứng dụng 29
A.3 Danh mục thẻ phân công và reviewer 29

Danh sách hình vẽ

3.1 Kiến trúc tổng thể của BrewLite 12
3.2 Máy trạng thái: nét liền là nhánh đã có; nét đứt là phần cần hoàn thiện 15

Danh sách bảng

1.1 Đối tượng và nhu cầu sử dụng BrewLite 2
1.2 Công nghệ đối chiếu từ cấu hình và mã nguồn 2

2.1 Danh mục yêu cầu chức năng và User Story liên quan 9
2.2 Yêu cầu phi chức năng và tiêu chí kiểm chứng 10

3.1 Trách nhiệm các module và thành phần nghiệp vụ 13
3.2 Các nhóm thực thể trong schema hiện tại 13
3.3 REST API chính đối chiếu từ controller 15

4.1 Đối chiếu yêu cầu với hiện trạng mã nguồn 18

5.1 Ca kiểm thử, yêu cầu liên quan và kết quả cần kiểm tra 20
5.2 Số liệu kiểm thử được lưu trong audit ngày 03/11/2026 21

6.1 Vai trò Scrum và trách nhiệm điều phối 23
6.2 Phân công theo thành viên Nhóm 7 24
6.3 Điểm phối hợp chính trong luồng nghiệp vụ 25
6.4 Product Backlog tổng hợp theo tài liệu nhóm 26

A.1 Danh mục 40 thẻ, người phụ trách và người kiểm tra 29

Chương 1

Giới thiệu đề tài

1.1 Bối cảnh và lý do chọn đề tài

Tại quầy đồ uống, khách hàng thường cần xem menu, hỏi giá theo kích cỡ, chọn topping rồi chờ nhân viên ghi nhận và thanh toán. Khi nhiều đơn được đặt cùng lúc, thao tác thủ công dễ làm sai tùy chọn, nhầm tổng tiền hoặc nhận đơn vượt lượng nguyên liệu còn có thể phục vụ.

BrewLite là ứng dụng web hỗ trợ khách tự chọn đồ uống và thanh toán không tiền mặt theo luồng đặt món tại quầy. Một đơn hàng giữ được các tùy chọn và giá tại thời điểm đặt; hệ thống kiểm tra nguyên liệu trước khi xác nhận và cung cấp thông tin để khách theo dõi đơn. Đây cũng là bài toán để nhóm thực hành transaction, xử lý đồng thời và chống ghi nhận thanh toán trùng trong phạm vi môn Công nghệ phần mềm.

1.2 Mục tiêu

- Cung cấp luồng khách hàng từ xem menu, tùy chỉnh đồ uống, quản lý giỏ, đăng nhập, tạo đơn đến thanh toán và xem lại đơn.
- Xác nhận giá ở backend, lưu bản chụp thông tin món và bảo vệ dữ liệu đơn hàng theo tài khoản đăng nhập.
- Quản lý khả năng phục vụ theo nguyên liệu và công thức; giữ, sử dụng hoặc hoàn nguyên liệu theo vòng đời đơn hàng.
- Mô phỏng thanh toán Ví/Thẻ, hỗ trợ thử lại và xử lý một yêu cầu thanh toán lặp mà không tạo tác động tài chính trùng.
- Tổ chức phát triển theo Scrum, có phân công, tiêu chí chấp nhận, mã nguồn, kiểm thử và tài liệu bàn giao để đối chiếu.

1.3 Phạm vi và đối tượng sử dụng

Bảng 1.1: Đối tượng và nhu cầu sử dụng BrewLite

Đối tượng	Nhu cầu và ranh giới
Khách chưa đăng nhập	Xem menu, chi tiết và chuẩn bị giỏ hàng; cần đăng nhập trước khi gửi đơn.
Khách hàng (CUSTOMER)	Đặt món, thanh toán, xem xác nhận, lịch sử và chi tiết đơn của mình.
Nhân viên (BARISTA)	Nhu cầu nhận đơn, pha chế và giao món; tài khoản/role đã có, giao diện và API vận hành quầy còn trong kế hoạch.
Mock Payment Gateway	Tác nhân tích hợp trả kết quả thanh toán giả lập; không thực hiện giao dịch tiền thật.
Nhóm phát triển, giảng viên	Theo dõi backlog, đánh giá chức năng, chất lượng và kết quả thực hiện.

Phạm vi sản phẩm tập trung vào đồ uống tại một cửa hàng. Các nội dung như vận chuyển, đặt bàn, quản lý chuỗi cửa hàng, thanh toán qua ngân hàng thật và hoàn tiền thật không thuộc phần hiện thực đang báo cáo. Coupon và loyalty là nghiệp vụ hỗ trợ luồng mua hàng; sự tồn tại của bảng dữ liệu không đồng nghĩa đã có đầy đủ giao diện quản trị.

1.4 Công nghệ sử dụng

Bảng 1.2: Công nghệ đối chiếu từ cấu hình và mã nguồn

Thành phần	Công nghệ và vai trò
Frontend	Next.js, React, TypeScript, Tailwind CSS; các trang trong `apps/web/src/app/`.
Trạng thái giao diện	TanStack Query cho dữ liệu API; giỏ hàng dùng store tự xây dựng với `useSyncExternalStore` và `localStorage`.
Backend	NestJS, TypeScript, REST API; tách module Auth, Catalog, Orders, Inventory, Payments và Promotions.
Lưu trữ	PostgreSQL, Prisma ORM, migration có phiên bản; tiền lưu bằng số nguyên VND.
Xác thực	JWT, Passport, bcryptjs; kiểm tra role và quyền sở hữu tài nguyên.
Kiểm thử	Vitest ở backend, bộ kiểm thử Node ở frontend và Playwright cho trình duyệt.
Hạ tầng	Docker Compose, GitHub Actions; Node.js 24 theo cấu hình của hai ứng dụng.

1.5 Cơ sở xây dựng báo cáo

Báo cáo tham khảo cách trình bày User Story và Acceptance Criteria trong `assigment1_tham_khao.pdf`, đặc biệt sáu câu chuyện của luồng khách hàng [1]. Nội dung công nghệ, chức năng và phân công được đối chiếu lại với mã nguồn, schema, Product Backlog và hồ sơ nhóm [2, 5, 7]. Các mô tả cũ như dùng Zustand hoặc lưu tồn trực tiếp theo thành phẩm không được dùng để mô tả hiện trạng.

Báo cáo gồm sáu chương: giới thiệu; phân tích yêu cầu; thiết kế hệ thống; hiện thực; kiểm thử; tổ chức thực hiện và kết luận. Mã US, FR và NFR lần lượt nhận diện User Story, yêu cầu chức năng và yêu cầu phi chức năng, phục vụ truy vết giữa nhu cầu, cách hiện thực và kiểm chứng.

Chương 2

Phân tích yêu cầu

2.1 Quy ước và mức ưu tiên

Mỗi User Story theo mẫu: "Là một [vai trò], tôi muốn [hành động], để [giá trị]". Tiêu chí chấp nhận mô tả điều kiện có thể kiểm tra khi nghiệm thu. **Must** là yêu cầu cốt lõi của luồng đặt món; **Should** là nghiệp vụ nâng cao cần hoàn thiện; **Extension** là phần mở rộng. Mức ưu tiên thể hiện nhu cầu, không xác nhận đã triển khai. Hiện trạng được tổng hợp ở Bảng 4.1.

2.2 Danh sách User Story và tiêu chí chấp nhận

2.2.1 US-01: Xem danh sách đồ uống

User Story – Must. Là một khách hàng, tôi muốn xem danh sách đồ uống để lựa chọn món phù hợp.

Tiêu chí chấp nhận:

AC-1: Menu hiển thị tên, hình ảnh và giá của sản phẩm đang được bán; dữ liệu được lấy từ Catalog API.

AC-2: Khi đang tải, chưa có sản phẩm hoặc API lỗi, giao diện hiển thị trạng thái tương ứng và cho phép thử lại khi phù hợp.

AC-3: Chọn một sản phẩm mở chi tiết có các kích cỡ và topping hợp lệ.

2.2.2 US-02: Tùy chỉnh đồ uống

User Story – Must. Là một khách hàng, tôi muốn chọn kích cỡ, topping, đường, đá và ghi chú để đồ uống phù hợp sở thích của mình.

Tiêu chí chấp nhận:

AC-1: Chỉ chọn được biến thể và topping được gán cho sản phẩm, còn hoạt động và đủ điều kiện phục vụ.

AC-2: Giá tạm tính cập nhật theo kích cỡ, topping và số lượng; đường, đá và ghi chú không làm thay đổi giá.

AC-3: Đường/đá có lựa chọn công thức chuẩn hoặc 0/25/50/75/100%; giá trị 0 được giữ đúng, không thay bằng mặc định.

AC-4: Ghi chú được trim; rỗng thành không có ghi chú, tối đa 500 ký tự. Tùy chọn đã chọn được chuyển đầy đủ vào giỏ.

2.2.3 US-03: Quản lý giỏ hàng

User Story – Must. Là một khách hàng, tôi muốn thêm, sửa số lượng và xóa món trong giỏ để điều chỉnh đơn trước khi đặt.

Tiêu chí chấp nhận:

AC-1: Thêm món đã tùy chỉnh, tăng/giảm số lượng, sửa tùy chọn và xóa dòng hàng; số lượng và tổng tạm tính cập nhật cùng nhau.

AC-2: Dòng có cùng sản phẩm, size, topping, đường, đá và ghi chú được nhận diện nhất quán; khác tùy chọn hoặc ghi chú được giữ thành dòng riêng.

AC-3: Giỏ được khôi phục sau khi tải lại trang; đọc được dữ liệu giỏ cũ thiếu trường tùy chọn theo cơ chế chuẩn hóa của store.

AC-4: Giỏ rỗng có thông báo và không cho gửi yêu cầu tạo đơn rỗng.

2.2.4 US-04: Tạo đơn hàng

User Story – Must. Là một khách hàng đã đăng nhập, tôi muốn gửi giỏ hàng để nhận mã đơn và số tiền do hệ thống xác nhận trước khi thanh toán.

Tiêu chí chấp nhận:

AC-1: Chỉ CUSTOMER đang hoạt động được tạo đơn; chủ đơn lấy từ JWT, không lấy từ mã người dùng trong nội dung request.

AC-2: Request có 1–20 dòng, mỗi dòng có số lượng nguyên 1–20; UUID, size, topping, đường/đá và ghi chú phải hợp lệ.

AC-3: Backend đọc giá hiện hành, tự tính tổng và thẩm định coupon nếu có; không dùng đơn giá hoặc tổng tiền do trình duyệt quyết định.

AC-4: Trong một transaction, hệ thống lưu đơn `PENDING`, chi tiết món dạng snapshot, giữ nguyên liệu, giữ lượt coupon nếu có và ghi nhận mốc khởi tạo đơn.

AC-5: Thiếu nguyên liệu hoặc lỗi bất kỳ bước ghi nào phải rollback, không để lại đơn hay reservation dở dang.

AC-6: Thành công trả mã đơn, tổng tiền và thời hạn giữ chỗ để tiếp tục thanh toán; dữ liệu không hợp lệ trả lỗi có thể xử lý trên giao diện.

2.2.5 US-05: Thanh toán không tiền mặt và thử lại

User Story – Must. Là một khách hàng, tôi muốn thanh toán bằng Ví hoặc Thẻ qua cổng giả lập để hoàn tất đơn mà không dùng tiền mặt.

Tiêu chí chấp nhận:

AC-1: Khách chọn phương thức thanh toán; API xác thực chủ đơn và lấy số tiền phải trả từ đơn đã lưu.

AC-2: Request thanh toán có `Idempotency-Key`; gửi lặp cùng key và cùng nội dung không tạo thêm giao dịch hoặc cộng điểm lần nữa.

AC-3: Dùng lại key với nội dung khác bị từ chối; các yêu cầu đồng thời không được tạo nhiều lần thanh toán thành công cho cùng đơn.

AC-4: Thành công đưa đơn sang `PAID`, sử dụng reservation và chuyển sang màn hình xác nhận; giỏ chỉ được xóa sau thành công.

AC-5: Thất bại có thông báo và giữ giỏ; thử lại một lần thanh toán mới dùng key mới trên cùng đơn còn hợp lệ, không tạo lại đơn để thử lại.

AC-6: Đơn đã hết thời hạn giữ chỗ không được thanh toán như đơn còn hiệu lực.

2.2.6 US-06: Xác nhận và xem lịch sử đơn hàng

User Story – Must. Là một khách hàng đã đăng nhập, tôi muốn xem xác nhận và lịch sử đơn của mình để kiểm tra món đã đặt và tiến trình xử lý.

Tiêu chí chấp nhận:

AC-1: Màn hình xác nhận có mã đơn, món/tùy chọn, tổng tiền, kết quả thanh toán và trạng thái đơn.

AC-2: Lịch sử chỉ trả đơn của tài khoản hiện tại, sắp xếp mới nhất trước; hỗ trợ tải trang tiếp theo theo cursor của API.

AC-3: Mở chi tiết xem được snapshot món, đường/đá/ghi chú, payment và các mốc trạng thái đã lưu; đơn người khác không được đọc.

AC-4: Không có đơn, phiên hết hạn và lỗi API có trạng thái giao diện rõ ràng; token không hợp lệ không được tiếp tục dùng.

2.2.7 US-07: Đăng ký, đăng nhập và kết thúc phiên

User Story – Must. Là một khách hàng, tôi muốn có tài khoản và đăng nhập để đặt món và quản lý đơn thuộc về mình.

Tiêu chí chấp nhận:

AC-1: Đăng ký kiểm tra dữ liệu và username trùng; mật khẩu chỉ được lưu dưới dạng hash, không trả hash trong response.

AC-2: Đăng nhập đúng cấp JWT; sai thông tin không cấp phiên. Khách tự đăng ký không được tự gán role BARISTA.

AC-3: Thiếu, sai hoặc hết hạn token bị từ chối ở API được bảo vệ; giao diện xóa phiên không hợp lệ và hướng người dùng đăng nhập lại.

AC-4: Đăng xuất xóa phiên ở trình duyệt; quyền đọc đơn vẫn được backend kiểm tra trong mọi request.

2.2.8 US-08: Áp dụng mã giảm giá

User Story – Should. Là một khách hàng, tôi muốn dùng coupon hợp lệ để nhận mức giảm phù hợp với đơn hàng.

Tiêu chí chấp nhận:

AC-1: Kiểm tra mã đang hoạt động, thời gian áp dụng, giá trị đơn tối thiểu, giới hạn tổng và giới hạn theo người dùng.

AC-2: Số tiền giảm đúng loại phần trăm/số tiền cố định, tuân theo mức trần và không làm tổng tiền âm.

AC-3: Kiểm tra trước chỉ là báo giá; khi tạo đơn phải kiểm tra và giữ lượt trong transaction dựa trên tổng do server tính.

AC-4: Thanh toán thành công sử dụng lượt; hủy/hết hạn giải phóng lượt theo vòng đời redemption; không vượt giới hạn khi có yêu cầu đồng thời.

2.2.9 US-09: Đặt món trong giới hạn nguyên liệu

User Story – Must. Là một khách hàng, tôi muốn chỉ đặt được số lượng có thể pha chế để đơn được tiếp nhận phù hợp với nguyên liệu thực tế.

Tiêu chí chấp nhận:

AC-1: Khả năng phục vụ của size được tính từ công thức và nguyên liệu; topping cũng tiêu hao nguyên liệu theo công thức riêng.

AC-2: Cộng gộp nhu cầu nguyên liệu dùng chung của các món và topping trước khi kiểm tra và giữ chỗ.

AC-3: Hai đơn đồng thời không làm tồn nguyên liệu âm; gặp xung đột thì thử lại có giới hạn hoặc trả lỗi phù hợp.

AC-4: Hủy/hết hạn hoàn đúng lượng đã ghi trong reservation, không tính lại từ công thức có thể đã thay đổi; mỗi reservation chỉ hoàn một lần.

2.2.10 US-10: Ghi nhận điểm thưởng

User Story – Should. Là một khách hàng, tôi muốn đơn thanh toán thành công được ghi nhận điểm thưởng để quyền lợi của mình được theo dõi chính xác.

Tiêu chí chấp nhận:

AC-1: Chỉ cộng điểm khi đơn thực sự được xác nhận đã thanh toán; thanh toán thất bại không tăng số dư.

AC-2: Một đơn chỉ tạo một giao dịch EARN dù replay hoặc có nhiều yêu cầu thanh toán đồng thời; số dư và ledger được cập nhật nhất quán.

AC-3: Chính sách quy đổi phải được nhóm xác nhận khi nghiệm thu. Code hiện tính `floor(total/10)`; báo cáo không coi tỷ lệ này là quy tắc đã được phê duyệt.

2.2.11 US-11: Nhân viên tiếp nhận và hoàn tất đơn

User Story – Extension cho giao diện quầy. Là một Barista, tôi muốn nhận đơn đã thanh toán và cập nhật quá trình pha chế để khách biết khi nào có thể nhận món.

Tiêu chí chấp nhận:

AC-1: Chỉ BARISTA được thao tác nghiệp vụ quầy; CUSTOMER bị từ chối.

AC-2: Chỉ đi theo thứ tự `PAID` → `PREPARING` → `READY` → `COMPLETED`; chặn bỏ bước và thao tác trên trạng thái kết thúc.

AC-3: Mỗi lần chuyển trạng thái lưu người thao tác, trạng thái trước/sau và thời gian; danh sách quầy hiển thị tùy chọn và ghi chú của món.

Ranh giới: máy trạng thái đầy đủ thuộc công việc core A1-10.2; page/queue của quầy là extension EXT-UC10. Cả hai chưa có đầy đủ runtime trong mã nguồn được đối chiếu.

2.2.12 US-12: Hủy đơn và xử lý hết hạn giữ chỗ

User Story – Should. Là một khách hàng, tôi muốn đơn chưa được xử lý tiếp được hủy theo quy tắc rõ ràng để không giữ nguyên liệu vô thời hạn.

Tiêu chí chấp nhận:

AC-1: Job tự hủy đơn chưa thanh toán đã quá hạn và hoàn reservation nguyên liệu/coupon đúng một lần.

AC-2: Thiết kế cho phép chủ đơn hủy ở `PENDING`, `PAYMENT_FAILED` hoặc `PAID` trước khi nhận pha; phải kiểm tra chủ đơn và trạng thái trong transaction.

AC-3: `COMPLETED` và `CANCELLED` là trạng thái kết thúc. Hủy từ PAID không được diễn giải là đã hoàn tiền qua ngân hàng.

Hiện trạng: job hết hạn đã có; API khách chủ động hủy và toàn bộ luật hủy còn cần hoàn thiện.

2.3 Yêu cầu chức năng (Functional Requirements)

Bảng 2.1: Danh mục yêu cầu chức năng và User Story liên quan

Mã	Chức năng cần cung cấp	Truy vết
FR-01	Trả và hiển thị menu từ dữ liệu sản phẩm đang hoạt động; xử lý loading/empty/error.	US-01
FR-02	Trả chi tiết, chọn size/topping hợp lệ, đường/đá/ghi chú và cập nhật giá tạm tính.	US-02
FR-03	Thêm/sửa/xóa, gộp/tách dòng đúng tùy chọn, tính tổng và khôi phục giỏ sau tải lại.	US-03
FR-04	Đăng ký, đăng nhập, đọc hồ sơ, đăng xuất phía client và xử lý phiên hết hạn.	US-07
FR-05	Tạo đơn từ JWT và giỏ hợp lệ; tính giá server, lưu snapshot và trạng thái PENDING.	US-04
FR-06	Tính nhu cầu nguyên liệu cho đồ uống và topping; giữ chỗ trong transaction khi tạo đơn.	US-04, US-09
FR-07	Thanh toán Ví/Thẻ qua mock gateway, xử lý thành công, thất bại, replay và retry.	US-05
FR-08	Hiển thị xác nhận; trả lịch sử có phân trang và chi tiết đơn thuộc tài khoản hiện tại.	US-06
FR-09	Kiểm tra coupon; giữ, sử dụng và giải phóng lượt theo vòng đời đơn.	US-08
FR-10	Ghi ledger điểm và cập nhật số dư đúng một lần khi đơn được thanh toán.	US-10
FR-11	Tự hủy đơn quá hạn; hoàn nguyên liệu và giải phóng coupon theo dữ liệu reservation.	US-09, US-12
FR-12	Thực thi đầy đủ máy trạng thái; chặn bước sai và ghi lịch sử mọi lần chuyển.	US-05, US-11, US-12
FR-13	Khách chủ động hủy đơn ở các trạng thái được cho phép và xem kết quả hủy.	US-12
FR-14	Quầy nhân viên nhận đơn, cập nhật pha chế, sẵn sàng và giao món.	US-11 (mở rộng)

2.4 Yêu cầu phi chức năng (Non-functional Requirements)

Các yêu cầu dưới đây xác định thuộc tính chất lượng và cách đánh giá. Chỉ tiêu được ghi là *mục tiêu* cần được đo khi nghiệm thu; báo cáo không coi là kết quả đã đạt nếu chưa có bằng chứng.

Bảng 2.2: Yêu cầu phi chức năng và tiêu chí kiểm chứng

Mã	Thuộc tính	Tiêu chí đánh giá
NFR-01	Bảo mật tài khoản	Hash mật khẩu; kiểm tra JWT; không ghi secret vào mã nguồn/response. Kiểm thử thiếu, sai, hết hạn token và đăng ký role trái phép.
NFR-02	Phân quyền dữ liệu	CUSTOMER chỉ đọc/thao tác đơn của mình; BARISTA chỉ thực hiện tác vụ được cấp. Kiểm thử role và truy cập chéo tài khoản.
NFR-03	Toàn vẹn giao dịch	Tạo đơn/giữ nguyên liệu/coupon cùng transaction; lỗi giữa chừng không để lại dữ liệu dở dang; tồn không âm. Kiểm thử rollback và DB đồng thời.
NFR-04	Chống tác động lặp	Replay key không gọi gateway lại hoặc cộng điểm lại; không có hai payment thành công cho một đơn. Kiểm thử cùng key, khác key, payload khác và retry.
NFR-05	Hiệu năng	Mục tiêu đề xuất: API menu/lịch sử có p95 dưới 2 giây với 50 người dùng đồng thời trên môi trường đã công bố; đo riêng gateway và chưa khẳng định đạt qua unit test.
NFR-06	Dễ sử dụng	Luồng mua hàng có trạng thái tải/rỗng/lỗi, tổng tiền và thông báo rõ ràng; không xóa giỏ khi thanh toán thất bại. Kiểm tra thủ công và E2E.
NFR-07	Tương thích giao diện	Mục tiêu kiểm tra màn hình 390 px và 1366 px, trình duyệt Chromium hiện đại; không che nút thao tác hoặc tràn ngang. Chưa suy rộng thành mọi thiết bị.
NFR-08	Bảo trì và kiểm thử	Chia module, dùng DTO và migration có phiên bản; nghiệp vụ có unit/integration test, thay đổi tích hợp được review chéo.
NFR-09	Tái lập môi trường	Có env mẫu, secret bắt buộc, migration chạy lặp và hướng dẫn seed. Nghiệm thu bằng môi trường sạch; chỉ có Compose chưa đủ chứng minh bàn giao.
NFR-10	Khả năng truy vết	Snapshot giữ thông tin đơn cũ; tiền dùng số nguyên VND, thời gian lưu UTC; lịch sử trạng thái phải phản ánh đủ các bước thực tế.

2.5 Quy tắc nghiệp vụ

BR-1: **Giá đáng tin cậy:** frontend chỉ hiển thị tạm tính; backend xác nhận giá và coupon trước khi lưu đơn.

BR-2: **Kho theo nguyên liệu:** không lưu số lượng thành phẩm ở ProductVariant. Số ly có thể làm là giá trị tính từ công thức; đường/đá được lưu như tùy chọn, không tự suy ra một chính sách giảm định lượng nguyên liệu nếu công thức chưa quy định.

BR-3: **Giữ chỗ có hạn:** TTL mặc định trong cấu hình là 15 phút, có thể thay bằng biến môi trường; thanh toán và job hết hạn phải đồng bộ.

BR-4: **Thanh toán:** phân biệt replay cùng lần thử và retry lần mới; tiền thanh toán lấy từ đơn trong database.

BR-5: **Lưu vết:** đổi tên/giá catalog không sửa snapshot của đơn cũ; mọi lần chuyển trạng thái theo yêu cầu phải có history tương ứng.

2.6 Đặc tả luồng tạo đơn và thanh toán

Tác nhân: CUSTOMER và Mock Payment Gateway. **Tiền điều kiện:** đăng nhập hợp lệ, giỏ có món và API hoạt động.

1. Khách kiểm tra giỏ, chọn phương thức thanh toán và xác nhận.
2. Frontend gửi danh sách biến thể, số lượng, topping, tùy chọn và coupon nếu có đến `POST /orders`.
3. Backend xác thực, tính lại tiền và thực hiện transaction tạo đơn/giữ nguyên liệu; trả đơn PENDING cùng thời hạn giữ chỗ.
4. Frontend gửi `POST/payments` kèm mã đơn, phương thức và key của lần thử; backend kiểm tra chủ đơn, hạn và số tiền.
5. Gateway trả thành công: payment được ghi nhận, đơn sang PAID, reservation/coupon được sử dụng và loyalty được ghi một lần.
6. Frontend nhận kết quả, hiển thị xác nhận và cho xem lại đơn.

Luồng thay thế: thiếu nguyên liệu thì không tạo đơn; validation lỗi thì sửa dữ liệu; gateway thất bại thì giữ giỏ và cho thử lại trên đơn còn hiệu lực; mất kết nối khi chưa rõ kết quả phải đối chiếu lần thử cũ, không mặc định đã thất bại; hết TTL thì giải phóng giữ chỗ.

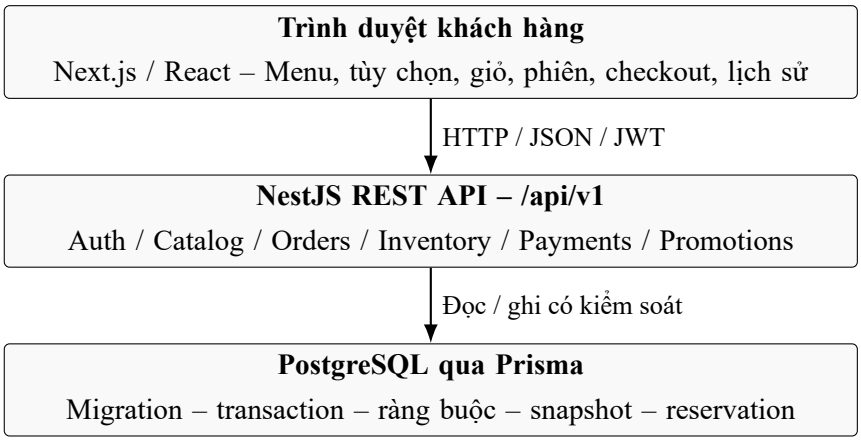
Hậu điều kiện mong đợi: thành công có đúng một đơn đã thanh toán và các tác động liên quan nhất quán; thất bại không để tồn âm, không cộng điểm và không tạo tác động thanh toán trùng.

Chương 3

Thiết kế hệ thống

3.1 Kiến trúc tổng thể

BrewLite tổ chức mã nguồn trong một repository, với frontend tại `apps/web/` và backend tại `apps/api/`. Backend là modular monolith: các module được triển khai chung trong một ứng dụng NestJS, sử dụng PostgreSQL làm nguồn dữ liệu nghiệp vụ.



Payments gọi Mock Payment Gateway;
OrderExpiryJob kiểm tra đơn quá hạn theo lịch.

Hình 3.1: Kiến trúc tổng thể của BrewLite

3.2 Phân chia trách nhiệm backend

Luồng phân lớp phổ biến là Controller → Service → Repository → Prisma → PostgreSQL. Controller tiếp nhận request, áp dụng guard/DTO và trả HTTP response; Service xử lý nghiệp vụ và điều phối transaction; Repository đóng gói truy vấn; Mapper chuyển đổi dữ liệu sang DTO.

Đây là định hướng tổ chức được thể hiện trong các module, không phải khẳng định mọi truy vấn đều đã đi qua Repository: `OrdersService` hiện còn trực tiếp dùng Prisma trong transaction tạo đơn, và `OrdersRepository.markAsPaid` chứa cả tác động coupon/loyalty. Việc đánh giá kiến trúc cần đối chiếu code thay vì chỉ dựa vào tên thư mục.

Bảng 3.1: Trách nhiệm các module và thành phần nghiệp vụ

Thành phần	Trách nhiệm
Auth / Users	Tài khoản, kiểm tra khách đang hoạt động, JWT, role và quyền sở hữu.
Catalog	Sản phẩm, size, topping hợp lệ, giá, công thức và khả năng phục vụ.
Orders	Tạo đơn, giá/snapshot, lịch sử/chi tiết, phối hợp transaction và hết hạn.
Inventory	Cộng gộp nhu cầu, giữ nguyên liệu, consume/release và kiểm soát version.
Payments	Payment attempt, request hash/key, admission, gateway và kết quả thanh toán.
Promotions	Thẩm định/khóa coupon và vòng đời giữ/sử dụng/giải phóng lượt.
Loyalty	Ledger và balance hiện được xử lý trong OrdersRepository; chưa có module độc lập.

3.3 Thiết kế cơ sở dữ liệu

Schema hiện tại có 18 model; nguồn đối chiếu là `apps/api/prisma/schema.prisma` và ERD cập nhật trong `docs/architecture/erd.md` [3, 4]. Các bảng được chia thành các nhóm sau.

Bảng 3.2: Các nhóm thực thể trong schema hiện tại

Nhóm	Model	Dữ liệu và quan hệ chính
Tài khoản	User	Username duy nhất, password hash, role CUSTOMER/BARISTA, trạng thái hoạt động và loyaltyBalance. Một user có nhiều order.
Danh mục	Product, ProductVariant, Topping, ProductTopping	Product có các size; bảng liên kết quy định topping được phép chọn cho từng sản phẩm; đơn giá dùng số nguyên.
Nguyên liệu và công thức	Ingredient, RecipeItem, ToppingRecipeItem	Ingredient có đơn vị G/ML/PCS, lượng tồn và version; công thức nối nguyên liệu với size hoặc topping.
Đơn và snapshot	Order, OrderItem, OrderItemTopping	Order lưu chủ đơn, tổng tiền, trạng thái và TTL; dòng món/topping giữ tên, giá và tùy chọn tại thời điểm đặt.
Giữ chỗ	InventoryReservation, InventoryReservationItem	Reservation gắn order/variant; chi tiết ghi đúng nguyên liệu và lượng đã trừ để hoàn kho.
Thanh toán và lịch sử	Payment, OrderStatusHistory	Payment có key, hash, phương thức và kết quả; history lưu trạng thái trước/sau, người đổi và thời gian.

Nhóm	Model	Dữ liệu và quan hệ chính
Khuyến mãi	Coupon, CouponRedemption	Coupon có loại giảm, thời hạn và giới hạn; redemption gắn user/order và vòng đời giữ lượt.
Điểm thưởng	LoyaltyTransaction	Ghi loại giao dịch, số điểm, số dư sau và liên kết order; dùng cùng balance trên User.

3.3.1 Kho nguyên liệu và công thức

Không lưu stock của thành phẩm ở ProductVariant. Với một biến thể có tập nguyên liệu I, số ly khả dụng được tính khi đọc:

$$\text{availableServings} = \min_{i \in I} \left\lfloor \frac{\text{stockQuantity}_i}{\text{recipeQuantity}_i} \right\rfloor .$$

Biến thể chưa có công thức hoặc sử dụng nguyên liệu ngừng hoạt động được coi là không thể phục vụ. Ví dụ, có 100 g cà phê và công thức cần 10 g/ly thì giới hạn từ nguyên liệu này là 10 ly; nếu sữa chỉ đủ 6 ly thì giới hạn chung là 6.

Topping có loại `PREPACKAGED` hoặc `PREPARED`, đều được biểu diễn bằng công thức tiêu hao nguyên liệu. Nhu cầu topping được cộng với nhu cầu đồ uống khi tạo đơn. Khả dụng từng size trên menu chỉ là thông tin tại thời điểm đọc; quyết định nhận cả giỏ phải kiểm tra lại trong transaction vì các món có thể dùng chung nguyên liệu.

3.3.2 Giữ, sử dụng và hoàn nguyên liệu

Khi tạo đơn, hệ thống trừ lượng nguyên liệu được giữ và ghi chi tiết vào reservation trong cùng transaction. Thành công thanh toán chuyển reservation sang CONSUMED; không trừ nguyên liệu lần thứ hai. Hủy hoặc hết hạn chỉ hoàn các reservation còn hiệu lực theo đúng lượng đã ghi.

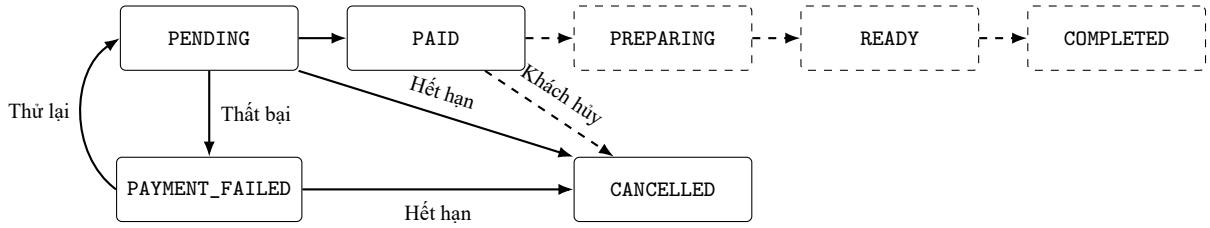
Cập nhật nguyên liệu kiểm tra đồng thời version và lượng tồn đủ. Lượng tồn không âm được bảo vệ thêm bằng CHECK trong database. Việc tạo đơn sử dụng mức cô lập Serializable và thử lại tối đa ba lần đối với một số xung đột được nhận diện.

3.3.3 Snapshot và ràng buộc dữ liệu

- Username, SKU, mã coupon và mã đơn có ràng buộc duy nhất; quan hệ bảng được bảo vệ bằng khóa ngoại.
- Size thuộc một sản phẩm là duy nhất; lượng công thức và reservation item phải dương; lượng tồn và giá không âm.
- Payment key duy nhất; partial unique index bảo vệ payment thành công/đang xử lý theo đơn và giao dịch EARN theo đơn.

- Tên món, size, topping và đơn giá được chụp vào đơn; đường/đá nullable có miền 0–100, ghi chú tối đa 500 ký tự.
- Tiền lưu kiểu số nguyên lớn theo VND và chuyển sang JSON có kiểm tra miền an toàn; thời gian lưu `timestamptz` theo UTC.

3.4 Máy trạng thái đơn hàng



Hình 3.2: Máy trạng thái: nét liền là nhánh đã có; nét đứt là phần cần hoàn thiện

Các trạng thái được khai báo trong enum không tự tạo ra hành vi chuyển trạng thái. Hiện luồng thanh toán, retry và tự hủy do hết hạn đã có code; nghiệp vụ pha chế/giao món và API khách hủy chưa triển khai đầy đủ. Thiết kế yêu cầu mọi bước phải có history. Việc thiếu mốc PAID trong `markAsPaid` là khoảng trống hiện trạng, được ghi ở Chương 5.

3.5 Thiết kế API hiện có

Tất cả endpoint dưới đây có tiền tố `/api/v1`. Bảng chỉ liệt kê route đã có controller; không tự bổ sung route hủy hoặc route Barista chưa tồn tại.

Bảng 3.3: REST API chính đối chiếu từ controller

Method	Endpoint	Quyền	Chức năng
GET	`/products`	Công khai	Menu và biến thể đang phục vụ.
GET	`/products/:id`	Công khai	Chi tiết và các topping hợp lệ.
POST	`/auth/register`	Công khai	Tạo tài khoản khách hàng.
POST	`/auth/login`	Công khai	Kiểm tra tài khoản và cấp JWT.
GET	`/auth/me`	JWT	Đọc hồ sơ tài khoản hiện tại.
POST	`/orders`	CUSTOMER	Tạo đơn, tính giá và giữ chỗ; thành công trả 201.
GET	`/orders/me`	CUSTOMER	Lịch sử theo tài khoản; limit, cursor và bộ lọc status.
GET	`/orders/:id`	Chủ đơn	Chi tiết, payment và status timeline.
POST	`/payments`	CUSTOMER, chủ đơn	Thanh toán; bắt buộc header Idempotency-Key.

Method	Endpoint	Quyền	Chức năng
POST	`/coupons/validate`	CUSTOMER	Kiểm tra coupon và trả báo giá.
GET	`/health`	Công khai	Kiểm tra API và khả năng truy cập database.

3.6 Thiết kế các màn hình chính

Luồng giao diện: `/menu` → modal chi tiết → CartDrawer → `/checkout` → `/orders/confirmation`. Các route `/login` và `/register` xử lý tài khoản; `/orders` và `/orders/[id]` cung cấp lịch sử/chi tiết.

Giỏ giữ trạng thái cục bộ để người dùng chưa đăng nhập vẫn chọn món. Khi checkout, phiên đăng nhập cung cấp token; giao diện gửi ID và tùy chọn thay vì tự quyết định tiền phải trả. Sau thất bại thanh toán, giỏ vẫn giữ nguyên; sau thành công, khách chuyển đến xác nhận theo mã đơn server đã tạo.

Chương 4

Hiện thực và đối chiếu mã nguồn

4.1 Tổ chức mã nguồn

```
apps/
  web/
    src/app/         # menu, login, register, checkout, orders
    src/features/    # auth, catalog, cart, orders, payments
    src/stores/      # cart-store.ts
    tests/           # Node tests va Playwright
  api/
    src/modules/     # auth, users, catalog, orders,
                     # inventory, payments, promotions, health
    prisma/          # schema, migrations, seed
    test/            # integration va HTTP E2E
scripts/             # kiem tra env, Compose va tai lieu
docs/                # API, ERD, Scrum, handover, evidence
```

Listing 4.1: Các thư mục phục vụ luồng nghiệp vụ đang có

4.2 Các điểm hiện thực chính

4.2.1 Catalog và giỏ hàng

Catalog đọc giá và quan hệ topping từ database; khả năng phục vụ được tính theo nguyên liệu thay vì số lượng thành phẩm. Modal chi tiết chuẩn hóa lựa chọn và chuyển vào store. Store dùng `useSyncExternalStore`, lưu `localStorage`, nhận diện dòng bằng đầy đủ tùy chọn và ghi chú, tránh gộp nhầm hai món khác sở thích.

4.2.2 Tạo đơn và kiểm soát đồng thời

`OrdersService.createOrder` kiểm tra khách đang hoạt động trong transaction, chuẩn hóa món, đọc catalog/công thức và tính tiền. Coupon được kiểm tra/khóa trước khi lưu; bản ghi order, item, topping, reservation và history khởi tạo được tạo trong cùng phạm vi transaction.

Lỗi tồn/version hoặc xung đột transaction có thể được thử lại có giới hạn. Kết quả thành công trả `PENDING`, mã đơn, số tiền VND và `reservationExpiresAt`. Giá và chủ đơn không lấy từ trường tùy ý do trình duyệt cung cấp.

4.2.3 Thanh toán, coupon và loyalty

`PaymentsController` yêu cầu JWT, role CUSTOMER, chủ đơn và key. `PaymentsService` điều phối payment attempt và mock gateway. Key cùng request hash nhận diện một lần thử; retry sau kết quả thất bại là lần thử mới. Khóa theo order giúp đồng bộ với job hết hạn và hạn chế xử lý hai lần thanh toán cho cùng đơn.

Khi xác nhận PAID, code hiện consume reservation nguyên liệu, consume coupon và cập nhật loyalty trong `OrdersRepository.markAsPaid`. Cơ chế ledger đã có thực thi; chưa có module loyalty tách riêng hoặc bằng chứng giao diện quản lý điểm hoàn chỉnh.

4.2.4 Phiên đăng nhập và quyền đọc đơn

Frontend có AuthSessionProvider, API client và xử lý logout/401. Backend bảo vệ route bằng JWT/role; danh sách đơn lọc theo tài khoản và chi tiết kiểm tra quyền sở hữu. Việc ẩn liên kết trên giao diện không thay cho kiểm tra ở backend.

4.2.5 Job hết hạn

`OrderExpiryJob` chạy mỗi phút, tìm đơn PENDING hoặc PAYMENT_FAILED đã quá hạn, khóa order và kiểm tra lại điều kiện. Job bỏ qua đơn có payment đang PROCESSING, đồng thời hủy đơn và hoàn các reservation hợp lệ trong transaction. Kiểm tra trạng thái trước khi cập nhật ngăn hoàn nguyên liệu lặp.

4.3 Ma trận yêu cầu và hiện trạng

Bảng dưới dựa trên controller/service hiện có và hồ sơ kiểm chứng ngày 03/11/2026 [8]. “Có triển khai” xác nhận có mã nguồn phù hợp, không thay cho nghiệm thu đủ Definition of Done hoặc mọi tiêu chí chấp nhận của User Story.

Bảng 4.1: Đối chiếu yêu cầu với hiện trạng mã nguồn

Yêu cầu	Hiện trạng	Căn cứ và giới hạn
FR-01–03	Có triển khai	Catalog API, modal, menu và cart store; có unit/contract và luồng browser checkout.
FR-04	Có triển khai	Auth API, session/logout/401, JWT/role/ownership; có unit/HTTP/browser session tests.
FR-05–06	Có triển khai	Tạo đơn Serializable, snapshot và giữ nguyên liệu; có DB integration/rollback/concurrency.
FR-07	Có triển khai, cần củng cố regression	Mock gateway/replay/retry đã có; một test idempotency cũ chỉ mock chính service; cần đưa probe DB thật vào bộ chạy thường xuyên.
FR-08	Có triển khai, timeline còn thiếu	Có confirmation/history/detail và API có phân trang; mốc PAID chưa được ghi history trong markAsPaid.

Yêu cầu	Hiện trạng	Căn cứ và giới hạn
FR-09	Có backend	Có validation/lifecycle coupon và tích hợp tạo đơn; chưa xác nhận giao diện coupon đầy đủ trong baseline được audit.
FR-10	Có backend, chờ chốt chính sách	Có balance/ledger khi PAID; tỷ lệ quy đổi hiện tại cần xác nhận với nhóm, trạng thái thẻ chưa phản ánh đầy đủ code.
FR-11	Có triển khai	Job TTL và release nguyên liệu/coupon; có kiểm thử database về expiry và race với payment.
FR-12	Một phần	Có payment/retry/expiry; chưa đầy đủ PREPARING, READY, COMPLETED; PAID history cần sửa.
FR-13	Chưa có API đầy đủ	OrdersController chưa có endpoint cho khách chủ động hủy.
FR-14	Chưa triển khai đầy đủ	Role BARISTA và route kiểm tra role đã có; chưa có page/queue và API vận hành quầy.

4.4 Cấu hình và triển khai

Docker Compose định nghĩa PostgreSQL, migrate, backend và frontend. Backend chờ database healthy và migrate thành công; frontend chờ backend healthy. `JWT_SECRET` là biến bắt buộc; cấu hình còn có issuer, audience, thời hạn JWT, chế độ gateway và TTL của order.

Migration service hiện chỉ triển khai schema, không tự seed menu. Do đó, bàn giao phải có bước seed và tài khoản demo rõ ràng. Quy trình host có thể tham khảo Phụ lục A. Báo cáo chưa khẳng định đã chạy sạch toàn bộ Compose hoặc đã triển khai production.

Chương 5

Kiểm thử và đánh giá

5.1 Chiến lược kiểm thử

BrewLite kết hợp kiểm thử đơn vị cho quy tắc và phép tính; kiểm thử integration với PostgreSQL cho ràng buộc, transaction và đồng thời; HTTP E2E cho API; Playwright cho luồng trình duyệt. Các kiểm tra lint, typecheck và build bổ sung tính nhất quán kỹ thuật, nhưng không thay thế kiểm tra tiêu chí chấp nhận.

Trong phạm vi chỉnh sửa báo cáo, nhóm số liệu dưới đây được sử dụng từ hồ sơ audit đã lưu ngày 03/11/2026 [8]. Báo cáo không ghi nhận đã chạy lại các bộ kiểm thử ứng dụng hoặc đã sửa lỗi runtime. Số liệu thuộc baseline của audit, trước thay đổi giao diện menu đang có trong working tree được hồ sơ đó nêu rõ.

5.2 Các ca kiểm thử tiêu biểu

Bảng 5.1: Ca kiểm thử, yêu cầu liên quan và kết quả cần kiểm tra

Mã	Tình huống / truy vết	Kết quả mong đợi và căn cứ
TC-01	Menu có dữ liệu, rỗng hoặc API lỗi. FR-01, NFR-06.	Hiển thị đúng trạng thái, có thử lại khi lỗi. Có catalog/menu-states tests.
TC-02	Chọn size/topping; đường/đá bằng 0; ghi chú rỗng/quá 500 ký tự. FR-02.	Giá đúng size/topping; giữ giá trị 0; chuẩn hóa/chặn dữ liệu sai. Có modal/DTO tests.
TC-03	Hai món khác đường/đá/ghi chú; sửa lượng và tải lại trang. FR-03.	Không gộp sai dòng, tổng đúng và dữ liệu giỏ khôi phục. Có cart-store tests.
TC-04	Register/login, username trùng, sai mật khẩu và token hết hạn. FR-04, NFR-01.	Hash mật khẩu, từ chối dữ liệu/phiên sai; client xử lý 401. Có auth unit/HTTP/session tests.
TC-05	Tạo đơn có dữ liệu/giá giả từ client; lỗi khi giữ nguyên liệu. FR-05–06, NFR-03.	Tiền do backend xác nhận; lỗi thì rollback đơn, chi tiết và giữ chỗ. Có orders DB integration.
TC-06	Hai giỏ đồng thời dùng cùng nguyên liệu cuối cùng. FR-06, NFR-03.	Tổng lượng nhận không vượt tồn; không tồn âm. Có inventory-concurrency integration.
TC-07	Thanh toán lỗi, sau đó thử lại cùng order. FR-07, NFR-06.	Giỏ còn nguyên khi lỗi; lần mới có key mới; thành công chỉ có một đơn được thanh toán. Có checkout browser test.

Mã	Tình huống / truy vết	Kết quả mong đợi và căn cứ
TC-08	Replay/cùng key đồng thời/khác payload/khác key cùng order. FR-07, NFR-04.	Không double-charge hoặc tác động lặp; sai payload bị từ chối. Probe DB trong audit có bằng chứng; cần regression chính thức đầy đủ.
TC-09	Coupon hết hạn/vượt giới hạn; release và retry. FR-09.	Không vượt lượt, không giảm âm; giữ và giải phóng nhất quán. Có promotions tests; cần củng cố toàn luồng fail/retry.
TC-10	Đơn hết TTL trong khi payment đang được xử lý. FR-11, NFR-03.	Job kiểm tra lại dưới khóa; không hoàn nguyên liệu đang được payment hợp lệ sử dụng. Có payment-expiry integration.
TC-11	Đọc chi tiết đơn tài khoản khác, history rỗng và phân trang. FR-08, NFR-02.	Chặn truy cập chéo, danh sách đúng chủ đơn và trạng thái rỗng rõ ràng. Có orders/session tests.
TC-12	Thành công/replay không nhân đôi điểm; timeline có mốc PAID. FR-10, FR-12, NFR-10.	Một EARN/đơn; mọi transition có history. Probe xác nhận EARN một lần nhưng phát hiện thiếu history PAID.
TC-13	Barista chuyển trạng thái sai thứ tự, khách chủ động hủy. FR-12–14.	Chặn quyền/trạng thái sai, ghi đủ history. Ca nghiệm thu còn phải bổ sung khi có runtime tương ứng.

5.3 Kết quả từ hồ sơ kiểm chứng

Bảng 5.2: Số liệu kiểm thử được lưu trong audit ngày 03/11/2026

Nhóm kiểm tra	Số test đạt	Phạm vi
Backend unit	213	29 file kiểm thử.
Frontend unit/component/contract	68	Giỏ, auth, catalog, menu và API contract.
Env và Compose config	9	Kiểm tra cấu hình.
API HTTP E2E	17	2 file; không coi toàn bộ là UI E2E.
PostgreSQL integration	53	8 file, kèm 10 migration chạy trên DB rỗng.
Chromium browser	9	2 checkout dùng API/DB thật; 7 session dùng API auth thật và mock response order.
Tổng	**369**	Tổng các nhóm nêu trên, không phải 369 E2E.

Hồ sơ còn ghi lint/typecheck hai ứng dụng, backend build và frontend build bằng Webpack trên Windows đạt; không xác nhận lại Turbopack mặc định. Migration deploy lần hai không thay đổi và seed được kiểm tra chạy lặp. Chưa có số liệu coverage đầy đủ, load test cho NFR-05 hoặc nghiệm thu giao diện trên mọi thiết bị.

Minh chứng lưu tại `docs/scrum/evidence/trello-audit-2026-11-03/`, gồm `backend-unit.txt`, `database-smoke.txt`, `browser-e2e.txt` và `payment-probe.txt`. Kết quả xanh của bộ hiện tại không bảo đảm phát hiện tất cả lỗi hoặc đáp ứng đủ DoD.

5.4 Vấn đề và giới hạn cần xử lý

1. **Lịch sử PAID:** code đổi trạng thái sang PAID nhưng `markAsPaid` chưa tạo `OrderStatusHistory` tương ứng. Cần cập nhật trong transaction và thêm assertion timeline.
2. **State machine và hủy:** chưa đủ bước pha chế/giao món và API khách hủy; enum/route kiểm tra role không thay thế nghiệp vụ.
3. **Idempotency regression:** file kiểm thử cũ mock chính `processPayment`; phải kiểm tra implementation với DB thật. Probe audit là bằng chứng bổ sung, chưa thay cho CI regression.
4. **Loyalty:** cần xác nhận tỷ lệ đổi điểm hiện tại và cập nhật nghiệm thu theo owner, không suy từ số lượng commit.
5. **Bàn giao:** cần demo toàn stack Compose sạch, bước seed, tài khoản demo và hồ sơ Sprint Review/Retro có minh chứng thực tế.

5.5 Điều kiện nghiệm thu

Theo Definition of Done của nhóm [9], một hạng mục cần build, lint/typecheck đạt, review chéo, kiểm thử theo mức rủi ro, kiểm tra AC, tài liệu/API/migration cập nhật và môi trường demo chạy được. Trạng thái Done trên board là thông tin theo dõi; chưa đủ để khẳng định mọi điều kiện nghiệm thu đã được chứng minh.

Chương 6

Tổ chức thực hiện và kết luận

6.1 Thành viên và vai trò Scrum

Nhóm 7 có năm thành viên. Vai trò được đối chiếu từ tài liệu phân công của nhóm [6, 7]; họ tên và MSSV được nhóm cung cấp. Phan Anh Khang kiêm Product Owner, Scrum Master và Tech Lead/Schema Owner; cả năm thành viên là Development Team và cùng chịu trách nhiệm về chất lượng sản phẩm.

Bảng 6.1: Vai trò Scrum và trách nhiệm điều phối

Vai trò	Phụ trách	Trách nhiệm
Product Owner	Phan Anh Khang	Ưu tiên backlog, làm rõ AC và điều phối nghiệm thu Increment.
Scrum Master	Phan Anh Khang	Điều phối Planning/Daily/Review/Retro, theo dõi blocker và cập nhật hồ sơ.
Tech Lead / Schema Owner	Phan Anh Khang	Kiến trúc, schema, hợp đồng và tích hợp. PR của Khang vẫn cần người khác review.
Development Team	Khang, Khải, Hoàng, Kiệt, Huy	Phân tích, thiết kế, code, kiểm thử, review chéo và bàn giao.

6.2 Bảng phân công công việc

Phân công theo mảng chính kết hợp chia đầu việc của từng luồng. Một module có thể được nhiều người phối hợp chỉnh sửa; người được giao chịu trách nhiệm đầu ra không nhất thiết là tác giả của mọi dòng code. Bảng 6.2 tổng hợp trách nhiệm, không quy đổi thành tỷ lệ đóng góp hoặc xác nhận tất cả đầu việc đã hoàn thành.

Bảng 6.2: Phân công theo thành viên Nhóm 7

Thành viên / MSSV	Mảng phụ trách chính	Đầu ra và phối hợp
Phan Anh Khang 3124410145	Nền tảng, schema/hợp đồng, Orders, tích hợp và Scrum.	Scaffold FE/BE, env/build/Compose; tạo đơn, snapshot, transaction, history/detail API; thiết kế state machine; phối hợp inventory/payment/E2E và hồ sơ Sprint.
Đoàn Quang Khải 3124410138	Frontend chi tiết, giỏ, checkout và màn hình sau thanh toán.	UI tùy chọn, cart store/identity/persist/tests; nối tạo đơn và payment; confirmation/history/detail; review các phần menu/auth/đầu ra Scrum theo thẻ.
Nguyễn Huy Hoàng 3124410105	Catalog, Inventory, dataset và seed.	Menu/detail API, menu UI và trạng thái; dữ liệu sản phẩm/size/topping; công thức và nguyên liệu; reserve/consume/release/concurrency; migration/seed demo.
Huỳnh Thái Kiệt 3124410172	Payments, Promotions và runtime state machine.	Mock gateway, payment outcome/retry/idempotency; coupon validation/lifecycle; phối hợp tác động PAID; full lifecycle core và extension staff queue theo kế hoạch.
Đỗ Mạnh Huy 3124410113	Auth, Security, Loyalty và tài khoản demo.	Register/login, bcrypt/JWT, session/logout/401; role/ownership; owner nghiệp vụ điểm thưởng, phối hợp PAID với Kiệt; hướng dẫn kiểm tra bảo mật và demo.

6.2.1 Phối hợp giữa các mảng

Bảng 6.3: Điểm phối hợp chính trong luồng nghiệp vụ

Luồng	Phân chia đầu việc
Menu và chi tiết	Hoàng: API/dữ liệu/menu; Khải: UI tùy chọn; Khang: schema và hợp đồng.
Giỏ và tạo đơn	Khải: giỏ/request; Khang: Orders; Huy: JWT/principal; Hoàng: nguyên liệu; Kiệt: coupon.
Thanh toán và retry	Kiệt: payment/gateway; Khải: checkout; Khang: tích hợp/E2E; Hoàng: reservation/expiry; Huy: phiên và quyền.
Điểm thưởng	Huy: owner nghiệp vụ; Kiệt: phối hợp tác động PAID trong code; Khang: reviewer theo phân công.
Lịch sử và xác nhận	Khang: API/dữ liệu; Khải: UI; Huy: phiên/ownership; Kiệt: payment/state; Hoàng: review UI.
Demo và bàn giao	Khang: toàn stack/tài liệu; Hoàng: migration/seed/menu; Huy: tài khoản/bảo mật; các thành viên kiểm tra chéo theo thẻ.

Theo bản phân công đối chiếu, 40 thẻ hiện hành gồm 36 thẻ kỹ thuật core, 3 thẻ Scrum và 1 extension. Số thẻ giao lần lượt là Khang 12, Hoàng 10, Huy 6, Khải 6, Kiệt 6. Kích thước thẻ khác nhau nên không dùng số thẻ để suy phần trăm công sức. Danh mục mã thẻ và reviewer được trình bày ở Phụ lục A.3.

6.3 Product Backlog và kế hoạch Sprint

Bảng sau giữ mã, Sprint và story point từ Product Backlog trong repository [5]. Đây là 10 hạng mục tổng hợp theo đề; bảng không thay thế danh mục thẻ chi tiết hoặc tự đổi lịch cam kết trên board.

Bảng 6.4: Product Backlog tổng hợp theo tài liệu nhóm

Mã	Sprint	Hạng mục / kết quả mong đợi	SP
BL-01	1	Khởi tạo monorepo, env mẫu, lint/test và README.	3
BL-02	1	API danh sách sản phẩm, seed và contract test.	3
BL-03	1	Menu tích hợp API, loading/empty/error.	3
BL-04	1	Chi tiết và size/topping/đường/đá/note hợp lệ.	5
BL-05	2	Giỏ thêm/sửa/xóa, identity, tổng và persist.	5
BL-06	2	Tạo đơn server-side, giữ tồn, trạng thái PENDING.	8
BL-07	2	Đăng ký/đăng nhập, hash, JWT và ownership.	5
BL-08	3	Thanh toán giả lập, idempotency, PAID/FAILED.	8
BL-09	3	Xác nhận, lịch sử, Docker và demo E2E.	8
BL-10	3	State machine, concurrency, coupon, loyalty và tests.	13

Mục tiêu Sprint 1 là nền tảng và menu; Sprint 2 là giỏ/tài khoản/tạo đơn; Sprint 3 là thanh toán và hoàn thiện nghiệp vụ. Board hiện được tổ chức thành S1 tuần 1–2, S2 tuần 3–4 và S3 tuần 5. Một số công việc như giỏ hoặc auth được lên thẻ ở mốc khác với bảng backlog tổng hợp. Báo cáo giữ sự phân biệt giữa hai cấp kế hoạch, không tự suy ngày họp, ngày kết thúc hoặc kết quả Sprint khi thiếu biên bản.

6.4 Kết quả đạt được

Đối chiếu source và hồ sơ kiểm chứng cho thấy luồng khách hàng menu, giỏ, đăng nhập, tạo đơn, thanh toán giả lập và xác nhận đã có tích hợp. Backend có tính giá và snapshot, cơ chế giữ nguyên liệu theo công thức, xử lý coupon, payment attempt và loyalty ledger. Lịch sử/chi tiết có API và giao diện gắn với tài khoản. Dự án có migration, seed và nhiều mức kiểm thử để kiểm tra các quy tắc quan trọng.

6.5 Hạn chế và hướng hoàn thiện

Ưu tiên tiếp theo là sửa lịch sử PAID; hoàn thiện state machine và luật hủy; đưa probe idempotency/loyalty vào regression dùng DB thật; chốt chính sách điểm; hoàn thiện bàn giao Compose/seed/tài khoản demo và hồ sơ Scrum. Giao diện quầy được phát triển như extension sau khi các quy tắc core được kiểm tra đầy đủ.

Các bước phát triển sau có thể bao gồm tích hợp cổng thanh toán thật, quy trình đối soát/hoàn tiền, đo tải theo NFR-05 và cải thiện trải nghiệm trên nhiều kích thước màn hình. Các nội dung này là định hướng, không được tính vào kết quả hiện tại của BrewLite.

Tài liệu tham khảo

[1] Tài liệu tham khảo nội bộ, *Assignment – BrewLite System*, 5 trang. Tệp `BrewLite-Report/tham_khao/assigment1_tham_khao.pdf`. Tham khảo bố cục User Story/Acceptance Criteria và mô tả kiến trúc; đã đối chiếu lại nội dung với dự án hiện hành.

[2] Nhóm BrewLite, *Mã nguồn frontend/backend và cấu hình dự án*, `apps/web/`, `apps/api/`, `compose.yaml`. Kho mã nguồn: https://github.com/akira-yami/BrewLite_A1. Nguồn sử dụng trong báo cáo là checkout local, đối chiếu ngày 03/11/2026.

[3] Nhóm BrewLite, *Prisma schema và migration*, `apps/api/prisma/schema.prisma`, `apps/api/prisma/migrations/`.

[4] Nhóm BrewLite, *ERD BrewLite – kho nguyên liệu và công thức*, `docs/architecture/erd.md`.

[5] Nhóm BrewLite, *Product Backlog BrewLite*, `docs/scrum/product-backlog.md`.

[6] Nhóm BrewLite, *Vai trò Scrum của nhóm BrewLite*, `docs/scrum/team-roles.md`.

[7] Nhóm BrewLite, *Phân công công việc và nghiệp vụ – đối chiếu 03/11/2026*, `docs/scrum/team-assignment-analysis-2026-11-03.md`. Đối chiếu board từ snapshot trong repository; không suy lại quyền sở hữu công việc theo tác giả commit.

[8] Nhóm BrewLite, *Đối chiếu BrewLite với Trello ngày 03/11/2026*, `docs/scrum/trello-audit-2026-11-03.md`, kèm các log tại `docs/scrum/evidence/trello-audit-2026-11-03/`.

[9] Nhóm BrewLite, *Definition of Done*, `docs/scrum/definition-of-done.md`.

Phụ lục A

Hướng dẫn chạy và phân công chi tiết

A.1 Chạy ứng dụng trên môi trường phát triển

Cần Node.js 24, PostgreSQL và thông tin kết nối riêng của môi trường. Các bước dưới đây dùng PowerShell; thực hiện từ thư mục gốc repository. Không ghi đè file env đang sử dụng. Nếu file đã có, kiểm tra và cập nhật các biến cần thiết theo env mẫu.

```
Set-Location apps/api
npm ci
# Chi sao chep khi chua co file .env
Copy-Item .env.example .env
# Dien DATABASE_URL va JWT_SECRET rieng trong .env.
# Tao database PostgreSQL truoc khi chay migrate.
npm run prisma:generate
npm run prisma:deploy
npm run db:seed
npm run start:dev
```

Listing A.1: Chuẩn bị backend và dữ liệu menu

Mở terminal khác để chạy frontend:

```
Set-Location apps/web
npm ci
npm run dev
```

Listing A.2: Chạy frontend từ gốc repository

Frontend mặc định tại `http://localhost:3000`, API có tiền tố `http://localhost:3001/api/v1`. Env frontend theo `.env.example`: cần kiểm tra `NEXT_PUBLIC_API_URL` khớp địa chỉ API. Kiểm tra `/health`, menu và đăng ký một tài khoản CUSTOMER trước khi demo. Không dùng seed E2E làm quy trình bàn giao chính thức hoặc coi seed catalog là đã tạo sẵn tài khoản Barista.

Compose có thể khởi chạy bằng `dockercomposeup--build` sau khi chuẩn bị env gốc. Service migrate hiện không seed catalog: phải thực hiện bước seed với cùng database qua môi trường API đã cấu hình, rồi kiểm tra menu. Hướng dẫn Compose sạch và tài khoản demo là hạng mục cần được nhóm kiểm chứng khi bàn giao.

A.2 Lệnh kiểm tra ứng dụng

```
# Trong apps/api
npm run lint
npm run typecheck
npm test
npm run test:e2e
# DB integration can SMOKE_DATABASE_URL cua DB test rieng.
npm run test:integration
npm run build

# Trong apps/web
npm run lint
npm run typecheck
npm test
npm run test:e2e
npm run build
```

Listing A.3: Các lệnh kiểm tra dùng trong từng ứng dụng

DB integration và browser E2E phải theo cấu hình fixture của repo và hướng dẫn `apps/api/README.md`, `tests/e2e/README.md`. Không trỏ các bài ghi dữ liệu vào DB đang dùng của ứng dụng. Kết quả của các lệnh này cần được lưu cùng mốc source khi thực sự chạy, không mặc định lấy từ bảng số liệu lịch sử.

A.3 Danh mục thẻ phân công và reviewer

Bảng trích từ bản đối chiếu phân công ngày 03/11/2026 [7]. Tên ngắn tương ứng các thành viên đầy đủ ở Bảng 6.2. Cột reviewer thể hiện người được giao kiểm tra, chưa xác nhận đã approve. Tên đầu việc giữ theo nội dung phân công; kỹ thuật cũ trên thẻ cần đọc cùng thiết kế/hiện trạng mới trong báo cáo.

Bảng A.1: Danh mục 40 thẻ, người phụ trách và người kiểm tra

Mã thẻ	Đầu việc được giao	Phụ trách	Reviewer
A1-1.1	Repository và khung API NestJS	Khang	Huy
A1-1.2	Khởi tạo Next.js và kết nối backend	Khang	Huy
A1-1.4	Env mẫu, lint/build và hướng dẫn môi trường	Khang	Huy
A1-2.1	Contract Product và GET /products	Khang	Kiệt
A1-10.1	Thiết kế Order State Machine	Khang	Kiệt
SCRUM-S1	Review/Retro và hồ sơ Sprint	Khang	Khải
A1-6.1	POST /orders, giá server và PENDING	Khang	Hoàng
A1-6.2	Order snapshot, transaction và rollback	Khang	Hoàng

Mã thẻ	Đầu việc được giao	Phụ trách	Reviewer
A1-9.1	API lịch sử và chi tiết đơn của tôi	Khang	Huy
SCRUM-S2	Review/Retro và hồ sơ Sprint	Khang	Khải
A1-9.5	Docker Compose và README bàn giao	Khang	Hoàng
SCRUM-S3	Review/Retro và hồ sơ Sprint	Khang	Khải
A1-1.3	Kết nối PostgreSQL/Prisma và dữ liệu mẫu	Hoàng	Khang
A1-2.2	Seed menu và API GET /products	Hoàng	Khải
A1-2.3	Kiểm thử API danh sách sản phẩm	Hoàng	Khải
A1-3.1	Lưới menu và thẻ đồ uống mobile-first	Hoàng	Khải
A1-3.2	Tích hợp API thật vào Menu	Hoàng	Khải
A1-3.3	Loading, empty, error và retry của Menu	Hoàng	Khải
A1-4.1	Product Detail API và tùy chọn hợp lệ	Hoàng	Khang
A1-4.3	Kiểm thử API tùy chọn và giá sản phẩm	Hoàng	Khang
A1-10.4	Inventory reserve/consume/release và concurrency	Hoàng	Khang
A1-9.3	Migration/seed và dữ liệu demo sạch	Hoàng	Khải
A1-7.1	Register/Login, bcrypt và JWT	Huy	Khang
A1-7.2	Auth UI, phiên, logout và 401	Huy	Khải
A1-6.3	JWT, validation và chủ sở hữu đơn	Huy	Khải
A1-7.3	Bảo vệ API đặt đơn và token an toàn	Huy	Khải
A1-10.6	Loyalty ledger/balance sau PAID	Huy	Khang
A1-9.4	Tài khoản demo, bảo mật và hướng dẫn test	Huy	Khang
A1-4.2	Detail UI, size/topping và giá tạm tính	Khải	Hoàng
A1-5.1	Cart store thêm/sửa/xóa và persist	Khải	Hoàng
A1-5.2	Định danh dòng giỏ và tính tổng	Khải	Hoàng
A1-5.3	Kiểm thử phép tính và khôi phục giỏ	Khải	Hoàng
A1-8.2	Checkout UI và xử lý payment outcome	Khải	Khang
A1-9.2	Confirmation và History/Detail UI	Khải	Hoàng
A1-10.5	Coupon validation và retry an toàn	Kiệt	Huy
A1-8.1	Mock gateway và POST /payments	Kiệt	Khang
A1-8.3	PAID/PAYMENT_FAILED qua OrdersService	Kiệt	Khang
A1-10.3	Payment idempotency và chống double-charge	Kiệt	Khang
A1-10.2	Hiện thực full Order State Machine	Kiệt	Khang
EXT-UC10	Staff UI queue và cập nhật trạng thái đơn	Kiệt	Huy